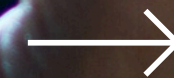


TIDY01: A Short Course in R Basics

Presented by Gabriel Rodrigues Palma



Tidyverse for Ecologists
(TIDY01)



Day 1 (9:30 – 17:30)

R Syntax & Variables
(9:35 – 11:05)

Loops & Functions
(13:30 – 15:00)

Data Types/Conditionals
(11:05 – 12:30)

Tidyverse for Ecologists
(TIDY01)



Loops & Functions

Tidyverse for Ecologists
(TIDY01)



Loops & Functions

Tidyverse for Ecologists
(TIDY01)



Loops & Functions

R Input

```
# Population growth simulation
initial_population <- 100
growth_rate <- 0.05
years <- 3
population <- initial_population

for (year in 1:years) {
  population <- population * (1 + growth_rate)
  cat("Year", year, ": Population =", round(population, 0), "\n")
}
```



Console Output

```
Year 1 : Population = 105
Year 2 : Population = 110
Year 3 : Population = 116
```


Loops & Functions

R Input

```
# Shannon diversity calculation function
calculate_shannon_diversity <- function(species_counts) {
  total_individuals <- sum(species_counts)
  proportions <- species_counts / total_individuals
  shannon_index <- -sum(proportions * log(proportions))
  return(shannon_index)
}

species_data <- c(45, 32, 28, 15)
diversity <- calculate_shannon_diversity(species_data)
cat("Shannon Diversity Index:", round(diversity, 3), "\n")
```



Console Output

```
Shannon Diversity Index: 1.324
```



Loops & Functions

R Input

```
# Temperature categorization loop
daily_temps <- c(17.5, 19.2, 21.8)
temp_categories <- character(length(daily_temps))

for (i in 1:length(daily_temps)) {
  if (daily_temps[i] < 18) {
    temp_categories[i] <- "Cool"
  } else if (daily_temps[i] < 21) {
    temp_categories[i] <- "Moderate"
  } else {
    temp_categories[i] <- "Warm"
  }
}
print(temp_categories)
```



Console Output

```
[1] "Cool"      "Moderate" "Warm"
```



Loops & Functions

R Input

```
# Logistic growth function
logistic_growth <- function(N0, r, K, time_steps) {
  population <- numeric(time_steps + 1)
  population[1] <- N0

  for (t in 1:time_steps) {
    population[t + 1] <- population[t] + r * population[t] *
      (1 - population[t] / K)
  }
  return(population)
}

deer_pop <- logistic_growth(N0 = 50, r = 0.1, K = 500, time_steps = 2)
cat("Population at time 2:", round(deer_pop[3], 1), "\n")
```

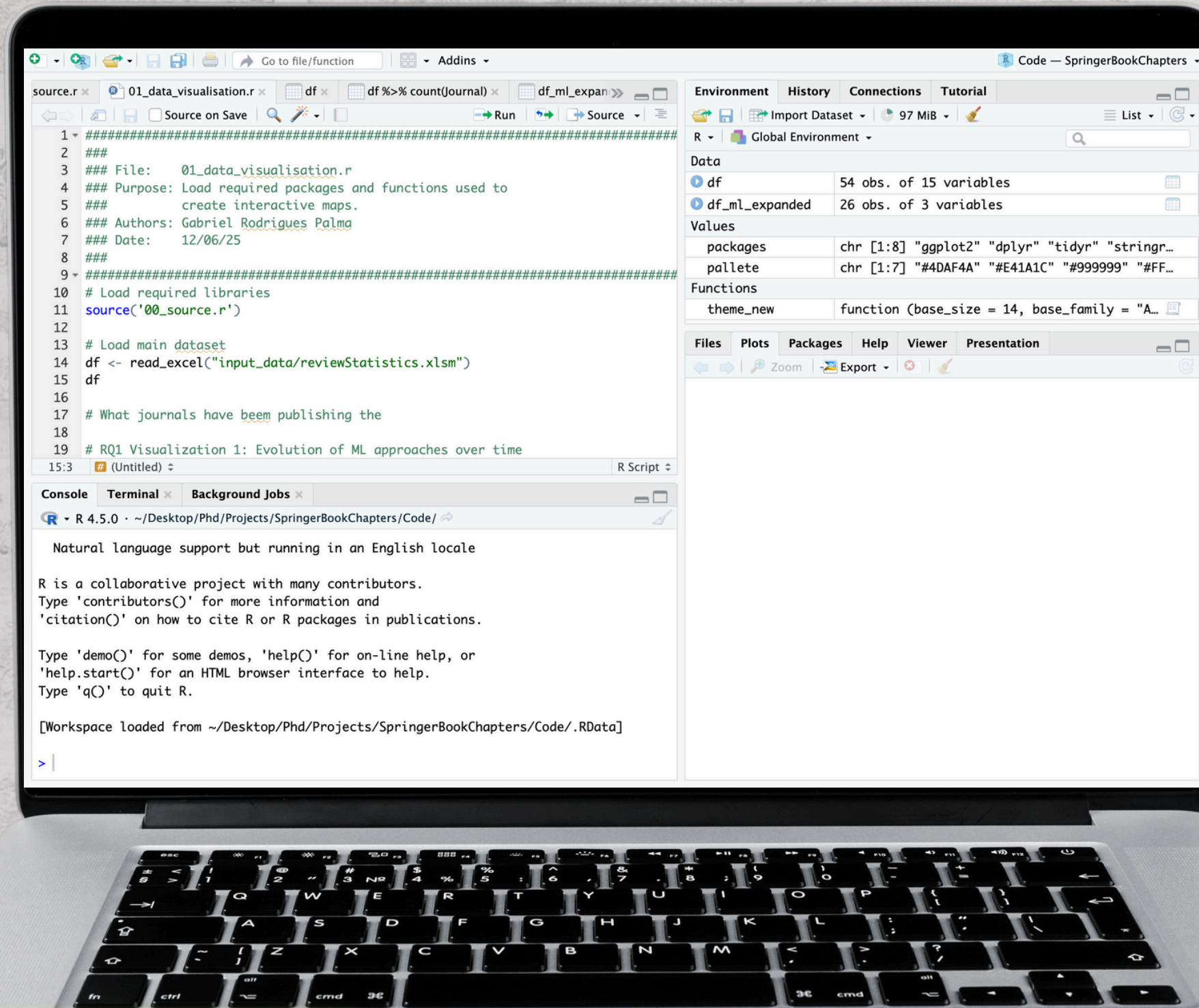


Console Output

```
Population at time 2: 59.4
```



Loops & Functions



Tidyverse for Ecologists
(TIDY01)



Day 1 (9:30 – 17:30)

R Syntax & Variables
(9:35 – 11:05)

Loops & Functions
(13:30 – 15:00)

Data Types/Conditionals
(11:05 – 12:30)

Data Structures & File
Handling (15:00 – 16:30)

Tidyverse for Ecologists
(TIDY01)



Data Structures & File Handling

Machine Learning
using Python (TIDY01)



Data Structures & File Handling

Data Structures & File Handling

R Input

```
# Species occurrence vector
species_list <- c("Quercus alba", "Acer rubrum", "Pinus strobus")
abundance_counts <- c(45, 32, 28)
names(abundance_counts) <- species_list
print(abundance_counts)
```



Console Output

Quercus alba	Acer rubrum	Pinus strobus
45	32	28



Data Structures & File Handling

R Input

```
# Ecological survey data frame
survey_data <- data.frame(
  site_id = c("SITE_01", "SITE_02", "SITE_03"),
  latitude = c(45.2386, 45.2401, 45.2358),
  species_count = c(24, 28, 19),
  canopy_cover = c(85.5, 78.2, 92.1)
)
head(survey_data, 2)
```



Console Output

```
  site_id latitude species_count canopy_cover
1 SITE_01  45.2386           24         85.5
2 SITE_02  45.2401           28         78.2
```



Data Structures & File Handling

R Input

```
# Multi-dimensional temporal data
temporal_data <- array(
  data = c(15, 20, 25, 18, 22, 28, 12, 16, 20, 14, 18, 24),
  dim = c(2, 3, 2),
  dimnames = list(
    sites = c("Site_1", "Site_2"),
    species = c("Sp_A", "Sp_B", "Sp_C"),
    time = c("2023", "2024")
  )
)
cat("Site_1, Species_A, 2023:", temporal_data["Site_1", "Sp_A", "2023"], "\n")
```



Console Output

```
Site_1, Species_A, 2023: 15
```



Data Structures & File Handling

R Input

```
# Species interaction matrix
species_names <- c("Species_A", "Species_B", "Species_C")
interaction_matrix <- matrix(
  c(0, 1, 0, 1, 0, 1, 0, 1, 0),
  nrow = 3, ncol = 3,
  dimnames = list(species_names, species_names)
)
print(interaction_matrix)
```



Console Output

	Species_A	Species_B	Species_C
Species_A	0	1	0
Species_B	1	0	1
Species_C	0	1	0



Data Structures & File Handling

R Input

```
# Nested ecosystem data list
ecosystem_data <- list(
  forest = list(
    dominant_species = c("Oak", "Maple"),
    avg_height = 25.5,
    biomass = 350.2
  ),
  grassland = list(
    dominant_species = c("Fescue", "Clover"),
    avg_height = 0.8,
    biomass = 12.5
  )
)
cat("Forest biomass:", ecosystem_data$forest$biomass, "\n")
```



Console Output

```
Forest biomass: 350.2
```



Data Structures & File Handling

R Input

```
survey_data <- data.frame(  
  site = c("Alpha", "Beta", "Gamma"),  
  abundance = c(32, 45, 27),  
  mean_temp = c(18.2, 19.7, 17.9)  
)  
abund_sub <- survey_data$abundance[2:3]  
row_gamma <- survey_data[survey_data$site == "Gamma", ]  
col_temp <- survey_data$mean_temp  
cat("Abundance subset:", abund_sub, "\n")  
print(row_gamma)  
cat("Mean temperature:", col_temp, "\n")
```



Console Output

```
Abundance subset: 45 27  
  site abundance mean_temp  
3 Gamma          27      17.9  
Mean temperature: 18.2 19.7 17.9
```


Data Structures & File Handling

R Input

```
habitats <- c("Forest", "Grassland", "Wetland", "Forest", "Wetland")
habitat_factor <- factor(habitats, levels = c("Forest", "Grassland", "Wetland"))
hab_counts <- table(habitat_factor)
cat("Factor levels:", levels(habitat_factor), "\n")
cat("Habitat counts:\n")
print(hab_counts)
```



Console Output

```
Factor levels: Forest Grassland Wetland
Habitat counts:
Forest      2
Grassland   1
Wetland     2
```


Data Structures & File Handling

R Input

```
biomass_matrix <- matrix(c(5.7, 8.4, 6.2, 9.1, 7.6, 8.0), nrow = 2, ncol = 3,  
  dimnames = list(c("SiteA", "SiteB"),  
    c("Species1", "Species2", "Species3")))  
row1 <- biomass_matrix[1, ]  
col3 <- biomass_matrix[, 3]  
submatrix <- biomass_matrix[1:2, 2:3]  
cat("Row 1:", row1, "\n")  
cat("Col 3:", col3, "\n")  
print(submatrix)
```



Console Output

```
Row 1: 5.7 8.4 6.2  
Col 3: 6.2 8.0  
      Species2 Species3  
SiteA      8.4      6.2  
SiteB      9.1      8.0
```


Data Structures & File Handling

R Input

```
count_vector <- c(12, 25, 17, 45, 8, 33)
third_item <- count_vector[3]
select_some <- count_vector[c(2, 4, 6)]
not_first <- count_vector[-1]
greater_than_20 <- count_vector[count_vector > 20]
cat("Third item:", third_item, "\n")
cat("Some elements:", select_some, "\n")
cat("Exclude 1st:", not_first, "\n")
cat(">20:", greater_than_20, "\n")
```



Console Output

```
Third item: 17
Some elements: 25 45 33
Exclude 1st: 25 17 45 8 33
>20: 25 45 33
```


Data Structures & File Handling

R Input

```
species_names <- c("Panthera leo", "Panthera tigris", "Panthera pardus")
observation_years <- c(2019, 2021, 2024)
population_trend <- c("increasing", "decreasing", "stable")
logical_vector <- c(TRUE, FALSE, TRUE)
cat("Species:", species_names, "\n")
cat("Years:", observation_years, "\n")
cat("Trends:", population_trend, "\n")
cat("Logical:", logical_vector, "\n")
```



Console Output

```
Species: Panthera leo Panthera tigris Panthera pardus
Years: 2019 2021 2024
Trends: increasing decreasing stable
Logical: TRUE FALSE TRUE
```


Data Structures & File Handling

source.r x 01_data_visualisation.r x df x df %>% count(Journal) x df_ml_expan

1 #####
2 ###
3 ### File: 01_data_visualisation.r
4 ### Purpose: Load required packages and functions used to
5 ### create interactive maps.
6 ### Authors: Gabriel Rodrigues Palma
7 ### Date: 12/06/25
8 ###
9 #####
10 # Load required libraries
11 source('00_source.r')
12
13 # Load main dataset
14 df <- read_excel("input_data/reviewStatistics.xlsm")
15 df
16
17 # What journals have been publishing the
18
19 # RQ1 Visualization 1: Evolution of ML approaches over time
15:3 (Untitled) R Script

Console Terminal Background Jobs
R - R 4.5.0 - ~/Desktop/Phd/Projects/SpringerBookChapters/Code/

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[Workspace loaded from ~/Desktop/Phd/Projects/SpringerBookChapters/Code/.RData]
> |

Code — SpringerBookChapters

Environment History Connections Tutorial
Import Dataset 97 MiB
Global Environment
Data
df 54 obs. of 15 variables
df_ml_expanded 26 obs. of 3 variables
Values
packages chr [1:8] "ggplot2" "dplyr" "tidyr" "stringr"
pallette chr [1:7] "#4DAF4A" "#E41A1C" "#999999" "#FF...
Functions
theme_new function (base_size = 14, base_family = "A...
Files Plots Packages Help Viewer Presentation
Zoom Export

Day 1 (9:30 – 17:30)

R Syntax & Variables
(9:35 – 11:05)

Loops & Functions
(13:30 – 15:00)

Clean Code in R
(16:30 – 17:30)

Data Types/Conditionals
(11:05 – 12:30)

Data Structures & File
Handling (15:00 – 16:30)

Tidyverse for Ecologists
(TIDY01)



Clean Code in R



Machine Learning
using Python (TIDY01)



Clean Code in R



Machine Learning
using Python (TIDY01)



Clean Code in R

Key differences that make the clean code version better:

- Meaningful variable names (plot_area vs p)
- Comments explaining each step
- Clear spacing and structure
- Descriptive output messages
- Easy to modify threshold values
- Logical organisation of calculation and classification

Both codes perform the same density calculation and classification, but the clean version is much easier to understand and maintain.

Clean Code in R

R Input

```
# Error handling and input validation
calculate_biodiversity_index <- function(community_data, index_type = "shannon")
  if (!index_type %in% c("shannon", "simpson", "richness")) {
    stop("index_type must be 'shannon', 'simpson', or 'richness'")
  }

  if (index_type == "richness") {
    return(sum(community_data > 0))
  } else if (index_type == "shannon") {
    proportions <- community_data / sum(community_data)
    return(-sum(proportions * log(proportions), na.rm = TRUE))
  }
}

test_data <- c(10, 0, 20, 5, 0, 15)
richness <- calculate_biodiversity_index(test_data, "richness")
cat("Species richness:", richness, "\n")
```



Console Output

```
Species richness: 4
```


Clean Code in R

R Input

```
# Modular diversity functions
diversity_functions <- list(
  shannon = function(abundances) {
    proportions <- abundances / sum(abundances)
    -sum(proportions * log(proportions), na.rm = TRUE)
  },
  simpson = function(abundances) {
    proportions <- abundances / sum(abundances)
    1 - sum(proportions^2, na.rm = TRUE)
  }
)

test_community <- c(15, 25, 35, 20)
shannon_div <- diversity_functions$shannon(test_community)
simpson_div <- diversity_functions$simpson(test_community)
cat("Shannon:", round(shannon_div, 3), "Simpson:", round(simpson_div, 3), "\n")
```



Console Output

```
Shannon: 1.351 Simpson: 0.736
```



Clean Code in R

R Input

```
# Meaningful constants and configuration
MIN_VIABLE_POPULATION <- 50
CRITICAL_HABITAT_THRESHOLD <- 0.3

assess_conservation_status <- function(population_size, habitat_remaining) {
  if (population_size < MIN_VIABLE_POPULATION) {
    status <- "Critical"
  } else if (habitat_remaining < CRITICAL_HABITAT_THRESHOLD) {
    status <- "Threatened"
  } else {
    status <- "Stable"
  }
  return(status)
}

status <- assess_conservation_status(75, 0.25)
cat("Conservation status:", status, "\n")
```



Console Output

```
Conservation status: Threatened
```


Clean Code in R

R Input

```
# Clear variable names and documentation
tree_diameter_cm <- 25
allometric_coefficient <- 0.15
estimated_biomass_kg <- tree_diameter_cm * allometric_coefficient
cat("Estimated biomass for", tree_diameter_cm, "cm")
cat("diameter tree:", estimated_biomass_kg, "kg\n")
```



Console Output

```
Estimated biomass for 25 cm
diameter tree: 3.75 kg
```



Clean Code in R

R Input

```
# Well-structured function with validation
calculate_species_evenness <- function(species_abundances) {
  if (length(species_abundances) == 0) {
    stop("Species abundances vector cannot be empty")
  }
  species_richness <- length(species_abundances)
  total_individuals <- sum(species_abundances)
  proportions <- species_abundances / total_individuals
  shannon_diversity <- -sum(proportions * log(proportions))
  evenness <- shannon_diversity / log(species_richness)
  return(round(evenness, 4))
}

test_data <- c(10, 20, 30, 15)
evenness <- calculate_species_evenness(test_data)
cat("Species evenness:", evenness, "\n")
```

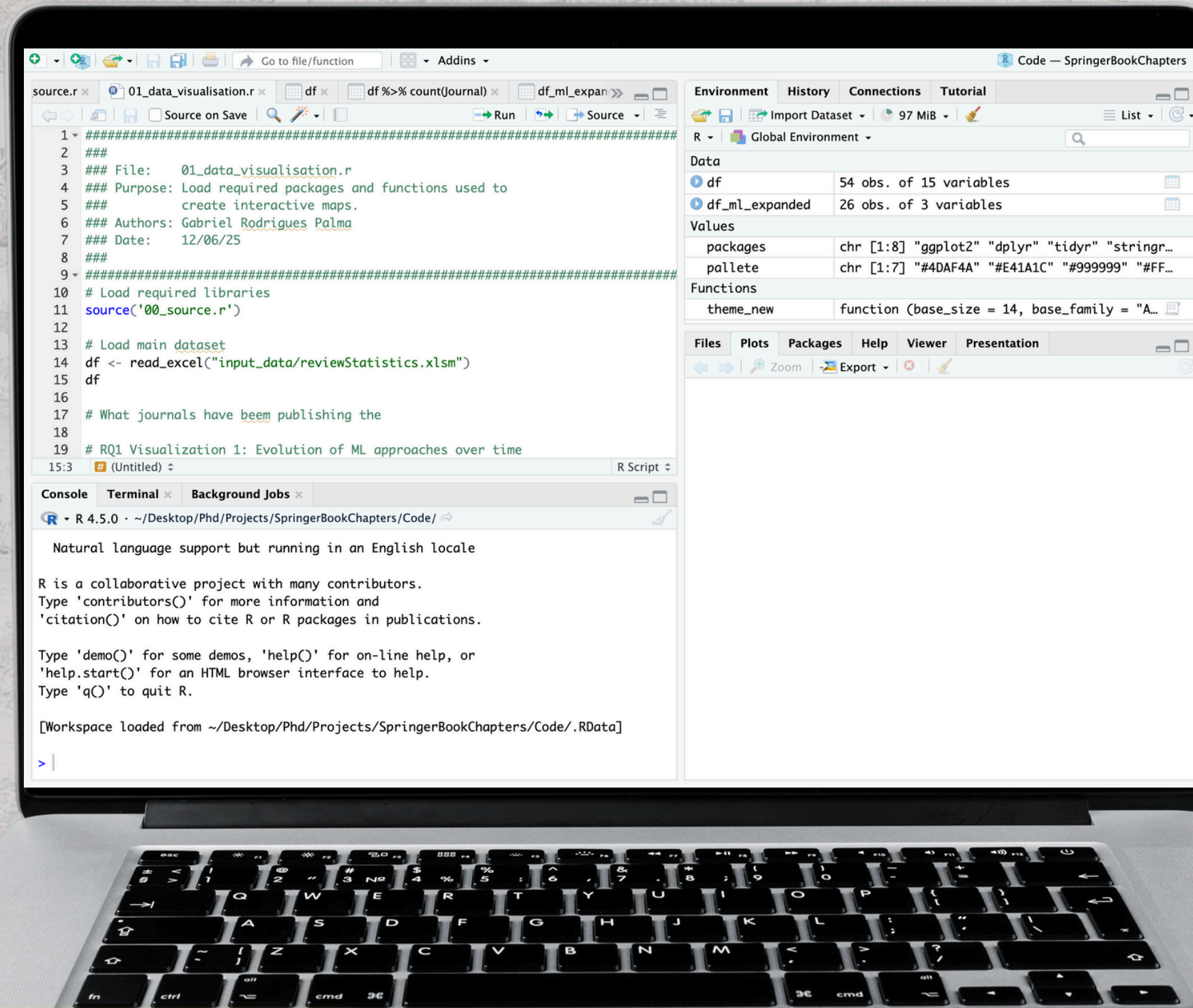


Console Output

```
Species evenness: 0.9477
```



Clean Code in R



Tidyverse for Ecologists
(TIDY01)

